a bit clearer

C PROGRAMMING FROM SCRATCH · EPISODE 07

Comments in C

Writing for Humans and Compilers

KEY POINTS

- Comments are stripped during preprocessing (replaced by a space) and carry zero runtime overhead.
- C provides two comment styles: classic block comments (/* ... */) and modern single-line comments (//).
- Block comments cannot be nested; attempting to nest them leads to premature closing and syntax errors.
- Good comments explain intentions and design decisions (the WHY), rather than self-evident mechanics.

01 Writing for Humans

▶ VIDEO 00:00

- When writing C code, you write for two audiences: the compiler (which demands exact syntax) and human readers (who need context and clarity).
- **Comments** are C's dedicated language mechanism for speaking directly to humans.

02 What Is a Comment?

▶ VIDEO 00:57

- A comment is explanatory text inserted into source code to explain logic, intent, or caveats.
- In **Translation Phase 3 (Preprocessing)**, the compiler removes every comment and replaces it with a single space (`' '`).
- **Zero Runtime Overhead**: Comments do not slow down execution, do not increase executable binary size, and consume zero memory at runtime.

03 Why We Use Comments

▶ VIDEO 01:52

1. **Explaining Code & Intent**: Clarifies tricky logic, non-obvious algorithms, and hardware constraints. Reflects engineering hygiene in teams and technical interviews.
2. **Temporary Debugging**: Temporarily comments out suspect lines or alternative implementations without deleting code.

04 The Two Comment Styles

▶ VIDEO 02:56

STYLE	SYNTAX	STANDARD	CHARACTERISTICS
Block Comment	`/* ... */`	ANSI C / C89	Can span multiple lines; can be embedded inline inside a statement (`int x = /* note */ 10;`).
Single-Line Comment	`// ...`	C99 (from C++)	Extends from `//` strictly to the end of the line; clean and compact.

05 The Nesting Trap

▶ VIDEO 04:15

Block comments **cannot be nested**. The compiler scanner matches the opening `/*` with the very first `*/` it encounters:

comments-demo.c — C SOURCE

```c
#include <stdio.h>

/* Multi-line block comment:
 * Explains comments in C. */
int main(void)
{
    // Print a greeting
    printf("Comments in C\n");

    int count = 10; /* inline note */
    printf("Count: %d\n", count);

    return 0;
}
```

KEY IDEA **Correct Usage**

Block comments document functions or sit inline; single-line comments document individual statements.

nested-comments.c — C SOURCE

```c
/* This file does not compile on purpose:
 * block comments cannot be nested */
#include <stdio.h>

int main(void)
{
    /* Outer comment begins
       /* Inner comment */
       residual text triggers a syntax error *
    return 0;
}
```

Output · Show output from: Build

```
1>nested-comments.c
1>nested-comments.c(9): error C2065:
'residual': undeclared identifier
1>nested-comments.c(9): warning C4138: '*/'
found outside of comment
========== Build: 0 succeeded, 1 failed
==========
```

KEY IDEA **Nesting Trap**

The inner `*/` terminates the comment early. The remaining text is exposed as invalid syntax, and the trailing `*/` causes a compiler error. Use `//` when commenting out code blocks.

06 Shortcuts in Visual Studio

▶ VIDEO 05:21

- **Comment selection**: `Ctrl+K, Ctrl+C` (inserts `//` at start of selected lines).
- **Uncomment selection**: `Ctrl+K, Ctrl+U` (removes `//` from start of selected lines).

07 Principles of Good Comments

▶ VIDEO 06:11

- **Explain WHY, not WHAT**: Code explains *how* the machine works; comments explain *why* the human chose that design. Avoid redundant comments like `i++; // add 1`.
- **Prefer Clear Names**: Do not use comments to compensate for poor variable or function names (`int days;` is clearer than `int d; // days`).
- **Keep Synchronized**: Always update comments when code changes; outdated comments are worse than no comments.

CODE 2 files on GitHub `github.com/abitclearer/c-programming-from-scratch/tree/main/07-comments-in-c`